BCA SEMESTER 5



Advanced DBMS

Subject Code: BCAMJ502

Course Credit: 02

Subject Faculty: Mr. Arjunsinh Vaghela

1. Introduction to NoSQL

What is NoSQL?

NoSQL stands for **"Not Only SQL."**

NoSQL databases are database management systems designed to store and process data in ways that are different from traditional relational databases.

Traditional databases such as MySQL, Oracle and PostgreSQL generally store data in:

- Tables
- Rows
- Columns
- Relationships

NoSQL databases can store data in different forms such as:

- Documents
- Key-value pairs
- Graphs
- Wide-column structures

Simple Example

Suppose we want to store information about a student.

In a relational database, we may have:

Student Table

Student_ID	Name	Course	Semester
101	Rahul	BCA	5
102	Priya	BCA	5

The structure of the table is predefined.

A NoSQL document database may store the same student as:

```
{
   "student_id": 101,
   "name": "Rahul",
   "course": "BCA",
   "semester": 5
}
```

Arjunsinh Vaghela

Another student may contain additional information:

```
{
   "student_id": 102,
   "name": "Priya",
   "course": "BCA",
   "semester": 5,
   "skills": ["Python", "Java", "SQL"],
   "phone": "9876543210"
}
```

Notice that the second document contains additional information without requiring us to modify a fixed table structure.

---

2. Why Was NoSQL Introduced?

To understand NoSQL, students should first understand the limitations of traditional relational databases when dealing with **modern large-scale applications**.

Earlier applications generally dealt with:

- Structured data
- Relatively smaller datasets
- Fixed database schemas
- Well-defined relationships

Modern applications generate enormous amounts of data from:

- Social media
- Mobile applications
- IoT devices
- E-commerce
- Online banking
- Streaming platforms
- Web applications
- Sensors
- Machine learning systems

For example:

Arjunsinh Vaghela

A social media application may generate:

User

↓

Posts

↓

Comments

↓

Likes

↓

Images

↓

Videos

↓

Followers

↓

Messages

The data is huge and changes continuously.

Therefore, organizations needed database technologies capable of providing:

- High scalability
- High availability
- Fast data access
- Flexible data models
- Distributed processing
- Handling of large volumes of data

This contributed to the growth of NoSQL databases.

---

3. Main Characteristics of NoSQL

NoSQL databases generally have several important characteristics.

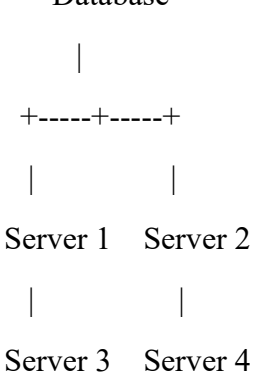
1. Flexible Schema

The structure of data does not necessarily have to be fixed before inserting data.

Arjunsinh Vaghela

2. Horizontal Scalability

The system can handle increasing workload by adding more machines/servers.



This is called **horizontal scaling**.

3. Distributed Architecture

Data can be distributed across multiple computers.

4. High Availability

Many NoSQL systems are designed to continue operating even if some servers fail.

5. Large Data Handling

NoSQL systems are commonly used for very large datasets and high-volume applications.

6. Different Data Models

NoSQL does not represent data only through relational tables.

---

4. Where is NoSQL Used?

NoSQL is particularly useful when applications require large-scale, distributed, flexible or rapidly changing data storage.

Major Application Areas

1. Social Media

Examples:

- User profiles
- Posts
- Comments
- Likes
- Followers
- Messages

Arjunsinh Vaghela

A social media application may have millions or billions of records.

---

2. E-Commerce

Examples:

- Product catalogs
- Shopping carts
- Customer preferences
- Product recommendations
- Orders
- Reviews

For example:

Product

├── Name

├── Price

├── Brand

├── Images

├── Specifications

├── Reviews

└── Offers

Different products may have different attributes.

A flexible data model can therefore be useful.

---

3. Internet of Things (IoT)

IoT devices continuously generate data.

For example:

Temperature Sensor

↓

25.4°C

↓

26.1°C

Arjunsinh Vaghela

↓

25.8°C

↓

27.2°C

Millions of devices can generate huge amounts of data.

---

4. Real-Time Applications

Examples:

- Real-time analytics
- Gaming
- Chat applications
- Recommendation systems
- Monitoring systems

---

5. Big Data

NoSQL databases are frequently used as part of large-scale data architectures.

---

6. Content Management

Websites may store:

- Articles
- Images
- Videos
- Comments
- Metadata

Different content types may have different attributes.

---

5. Types of NoSQL Databases

The four major categories students should remember are:

The four major types are:

1. Key-Value Database

Arjunsinh Vaghela

2. Document Database
3. Wide-Column Database
4. Graph Database

---

6. Key-Value Database

A key-value database stores data as:

KEY → VALUE

It is similar to a dictionary in programming.

Example

101 → Rahul

102 → Priya

103 → Amit

Or:

"user101" → "Rahul"

"user102" → "Priya"

The key uniquely identifies the value.

Structure

```
+----------+---------------+
| Key      | Value         |
+----------+---------------+
| user101  | Rahul         |
| user102  | Priya         |
| user103  | Amit          |
+----------+---------------+
```

Examples

- Redis
- Amazon DynamoDB

Applications

Key-value databases are useful for:

- Caching

Arjunsinh Vaghela

- Session management
- Shopping carts
- User preferences
- Fast lookups

---

7. Document Database

A document database stores data in documents.

Documents are commonly represented using formats such as:

- JSON
- BSON
- XML-like structures

Example

```
{
   "id": 101,
   "name": "Rahul",
   "course": "BCA",
   "skills": [
      "C",
      "Java",
      "Python"
   ]
}
```

Another document could be:

```
{
   "id": 102,
   "name": "Priya",
   "course": "BCA",
   "semester": 5,
   "city": "Anjar"
}
```

Arjunsinh Vaghela

The documents do not necessarily need exactly the same structure.

Examples

- MongoDB
- CouchDB

Applications

- E-commerce
- Content management
- User profiles
- Mobile applications
- Web applications

---

8. Wide-Column Database

Wide-column databases organize data using rows and flexible columns.

They are designed for large-scale distributed data processing.

Examples

- Apache Cassandra
- Apache HBase

Conceptually:

Row Key | Name | Course | City

---

101 | Rahul | BCA | Anjar

102 | Priya | BCA | Bhuj

Different rows may have different columns depending on the database model.

Applications

- Big Data
- IoT
- Large-scale analytics
- Distributed applications

---

9. Graph Database

Arjunsinh Vaghela

A graph database represents information using:

- Nodes
- Relationships/Edges
- Properties

Example

Suppose:

Rahul → FRIEND_OF → Amit

Amit → FRIEND_OF → Priya

Graphically:

[Rahul]

|

FRIEND_OF

|

[Amit]

|

FRIEND_OF

|

[Priya]

Here:

- Rahul = Node
- Amit = Node
- Priya = Node
- FRIEND_OF = Relationship

Examples

- Neo4j
- Amazon Neptune

Applications

Graph databases are particularly useful where relationships are important.

Examples:

- Social networks

Arjunsinh Vaghela

- Recommendation systems
- Fraud detection
- Network analysis
- Knowledge graphs

---

10. Comparison of NoSQL Types

Type	Basic Structure	Example	Common Use
Key-Value	Key → Value	Redis	Caching
Document	Document	MongoDB	Web applications
Wide-Column	Rows/Columns	Cassandra	Big Data
Graph	Nodes + Edges	Neo4j	Relationships

Easy Memory Trick

K-D-W-G

Key-Value
Document
Wide-Column
Graph

---

11. Why NoSQL?

This is one of the **most important examination questions**.

Major reasons for using NoSQL

1. Large Volume of Data

Modern applications generate huge amounts of data.

For example:

Users

↓

Millions

↓

Transactions

↓

Billions of records

NoSQL systems are designed for large-scale data workloads.

Arjunsinh Vaghela

2. High Velocity of Data

Data can be generated very quickly.

Examples:

- Sensor readings
- Online transactions
- Social media activity
- Website clicks

---

3. Variety of Data

Data may be:

- Structured
- Semi-structured
- Unstructured

Examples:

Text

Images

Videos

JSON

Sensor data

Logs

---

4. Flexible Schema

Suppose one product has:

Name

Price

Brand

Color

Another product may have:

Name

Price

Arjunsinh Vaghela

Brand

Storage

RAM

Processor

A document database can naturally represent these differences.

---

5. Horizontal Scalability

A database can scale by adding more machines.

Vertical Scaling

One powerful server

↓

More CPU

More RAM

More Storage

Horizontal Scaling

Server 1

Server 2

Server 3

Server 4

More servers are added to handle increasing workloads.

---

12. Advantages of NoSQL

1. Scalability

NoSQL systems can be designed to scale across multiple machines.

2. Flexibility

They can handle data whose structure changes over time.

3. High Performance

Certain NoSQL databases can provide very fast operations for appropriate workloads.

4. High Availability

Data can be replicated across multiple nodes in distributed systems.

Arjunsinh Vaghela

5. Large Data Handling

Suitable NoSQL technologies can handle very large datasets.

6. Distributed Processing

Data can be distributed among multiple servers.

7. Suitable for Big Data

NoSQL technologies are widely used in big-data architectures.

8. Supports Semi-Structured Data

For example:

```
{
  "name": "Rahul",
  "skills": ["Java", "Python"]
}
```

---

13. Disadvantages / Limitations of NoSQL

For a balanced understanding, students should also know the limitations.

1. Lack of Standardization

Different NoSQL databases use different approaches and query languages.

2. Complex Transactions

Some NoSQL systems historically provided weaker transaction capabilities than traditional relational systems, although modern NoSQL databases vary considerably and many support ACID transactions.

3. Data Consistency Trade-offs

Some distributed NoSQL designs may make trade-offs between consistency, availability and partition tolerance depending on their architecture.

4. Learning Curve

Developers familiar with SQL may need to learn new data models and query methods.

5. Not Suitable for Every Application

Traditional relational databases remain very suitable for many applications involving:

- Complex relationships
- Strong transactional requirements
- Structured data

Arjunsinh Vaghela

- Extensive joins

---

14. Use of NoSQL in Industry

NoSQL technologies are used in many industries and application architectures.

E-Commerce

Used for:

- Product catalogs
- Customer profiles
- Shopping carts
- Recommendations

Banking and Finance

Potential applications include:

- Fraud detection
- Transaction-related analytics
- Customer activity
- Risk analysis

The specific database choice depends heavily on transaction and consistency requirements.

Healthcare

Possible applications:

- Patient-related data
- Medical records
- Sensor data
- Healthcare analytics

Telecommunications

Used for:

- Call records
- Network monitoring

Arjunsinh Vaghela

- Customer activity
- Large-scale event data

Social Media

Used for:

- Profiles
- Posts
- Messages
- Activity streams
- Relationships

IoT

Used for:

- Sensor data
- Device information
- Monitoring
- Time-series workloads

---

15. SQL vs NoSQL

This is a **very important theory and examination topic**.

Feature	SQL	NoSQL
Full form	Structured Query Language	Not Only SQL
Data model	Relational	Non-relational/multiple models
Structure	Tables	Documents, key-value, graph, wide-column
Schema	Usually predefined	Often flexible
Relationships	Strong relational support	Depends on database type
Scaling	Traditionally vertical, though distributed SQL also exists	Often designed for horizontal scaling

Arjunsinh Vaghela

Feature	SQL	NoSQL
Query language	SQL	Database-specific APIs/query languages
Joins	Strong support	Often avoided or model-dependent
Transactions	Strong ACID support	Varies by system
Data type	Primarily structured	Structured/semi-structured/unstructured depending on system
Examples	MySQL, PostgreSQL, Oracle	MongoDB, Cassandra, Redis, Neo4j

Important Point for Students

Do **not** teach students that:

SQL = old and NoSQL = new and always better.

That is incorrect.

A better statement is:

SQL and NoSQL databases are designed for different requirements, and modern database systems often overlap in their capabilities.

---

16. SQL Example vs NoSQL Example

Suppose we want to store:

Student ID: 101

Name: Rahul

Course: BCA

Semester: 5

SQL

```
CREATE TABLE Student (
   Student_ID INT,
   Name VARCHAR(50),
   Course VARCHAR(20),
   Semester INT
);
```

Insert:

```
INSERT INTO Student
```

Arjunsinh Vaghela

VALUES (101, 'Rahul', 'BCA', 5);

NoSQL Document

```
{
   "student_id": 101,
   "name": "Rahul",
   "course": "BCA",
   "semester": 5
}
```

The fundamental difference is the **data model and how the application interacts with the data**.

---

17. What is NewSQL?

Now comes an important modern database concept.

Definition

NewSQL refers to a class of modern relational database systems that aim to provide:

- SQL interface
- Relational data model
- ACID transactions
- Horizontal/distributed scalability

In simple words:

NewSQL tries to combine important strengths of traditional SQL databases with the scalability expected from modern distributed systems.

---

18. Why NewSQL?

Traditional SQL databases:

Excellent relational model

+

Strong transactions

+

SQL

Arjunsinh Vaghela

But very large distributed applications may require:

SQL

+

ACID

+

Horizontal scalability

+

Distributed architecture

NewSQL systems attempt to provide these capabilities together.

---

19. SQL vs NoSQL vs NewSQL

Feature	**SQL**	**NoSQL**	**NewSQL**
Data model	Relational	Non-relational/multiple models	Relational
Query	SQL	Varies	SQL
Schema	Usually fixed	Often flexible	Generally structured
ACID	Strong support	Varies	Strong support
Horizontal scaling	Varies	Common design goal	Major design goal
Distributed architecture	Possible	Common	Common
Complex relationships	Strong	Depends on type	Strong
Best suited for	Traditional relational applications	Large-scale/flexible/distributed workloads	Distributed relational applications
Examples	MySQL, PostgreSQL	MongoDB, Cassandra, Redis	CockroachDB, Google Cloud Spanner

---

Arjunsinh Vaghela

What Is MongoDB?

Definition: MongoDB is an open-source, document-oriented NoSQL database management system that stores data in flexible, JSON-like documents represented internally using BSON (Binary JSON).

MongoDB is used to store, retrieve, update, and delete data efficiently. It is particularly useful for applications that handle large amounts of data, require flexible data structures, or need to scale across multiple servers.

Unlike a traditional relational database, which organizes data into tables containing rows and columns, MongoDB organizes data into **collections and documents**.

Example: Student Record in MongoDB

Consider the following student information:

```
{
  "rollno": 101,
  "name": "Rahul",
  "course": "BCA",
  "semester": 5,
  "city": "Anjar"
}
```

This represents one student document.

Here:

- `rollno` is a field.
- `101` is the value of that field.
- `name`, `course`, `semester`, and `city` are also fields.
- The complete object represents a document.

MongoDB can store multiple such documents in a collection called `students`.

Brief History of MongoDB

MongoDB was developed by the company 10gen, which later became MongoDB, Inc.

Its first public release was in 2009. The database was designed to support flexible document storage, developer-friendly data handling, and distributed database capabilities.

The name MongoDB is commonly associated with the word humongous, reflecting the goal of handling large amounts of data.

MongoDB has since become a widely used database in web applications, cloud-based systems, analytics platforms, and other software applications.

Arjunsinh Vaghela

Basic MongoDB Terminology

Students should understand these terms before learning MongoDB queries.

Term	Meaning	Example
Database	An organized container for collections	`college`
Collection	A group of documents	`students`
Document	A single record containing fields and values	One student's details
Field	A named property in a document	`name`
Value	The data associated with a field	`"Rahul"`
BSON	Binary representation of JSON-like documents, used by MongoDB	A stored student document
`_id`	Unique identifier field for each document	`ObjectId("...")`
Query	An instruction to retrieve or process data	Find students from Anjar
Index	A data structure that helps MongoDB locate documents efficiently	Index on `rollno`

Hierarchy of MongoDB

The general structure is:

```
MongoDB Server
    |
    └── Database: college
            |
            ├── Collection: students
            |       |
            |       ├── Document 1
            |       ├── Document 2
            |       └── Document 3
            |
            └── Collection: teachers
                    |
                    ├── Document 1
                    └── Document 2
```

Remember:

- A database contains collections.
- A collection contains documents.
- A document contains fields and values.

Why Do We Use MongoDB?

MongoDB is used because it offers a flexible document model, powerful querying features, indexing, replication, and horizontal scaling.

Consider an online shopping application. Each product may have different properties.

Arjunsinh Vaghela

Advantages of MongoDB

1. Flexible Schema

MongoDB does not require every document in a collection to have exactly the same fields.

Developers can accommodate changing data requirements without necessarily redesigning a

2. Document-Oriented Storage

MongoDB stores related information in documents.

Related information can often be retrieved together without requiring multiple joins.

3. Supports Different Data Types

MongoDB supports values such as:

- Strings
- Numbers
- Booleans
- Arrays
- Embedded documents
- Dates
- Null values
- ObjectIds

4. High Availability Through Replication

MongoDB supports replication using **replica sets**.

Replication can improve availability and help protect against certain server failures.

5. Indexing Support

MongoDB supports indexes on fields.

For example:

```
db.students.createIndex({rollno: 1})
```

An index on `rollno` can help queries that search by roll number.

Arjunsinh Vaghela

Appropriate indexes can improve query performance.

6. Aggregation

MongoDB provides aggregation pipelines to calculate totals, averages, counts, and grouped results.

7. Developer-Friendly Data Model

MongoDB documents resemble JavaScript objects and JSON data.

This can make MongoDB convenient for applications that already work with JSON-like data structures.

Limitations of MongoDB

1. Data Duplication
2. Schema Design Requires Planning
3. Not Ideal for Every Workload
4. Operational Complexity

USING JSON IN MONGODB

What Is JSON?

JSON stands for JavaScript Object Notation.

JSON is a lightweight, text-based format used to represent and exchange data between applications.

It represents data using key-value pairs and arrays.

JSON is widely used in web applications, APIs, configuration files, and data exchange.

Arjunsinh Vaghela

MongoDB uses a document model that resembles JSON.

For example, an application can send student data in JSON format:

```
{
  "rollno": 101,
  "name": "Rahul",
  "course": "BCA"
}
```

MongoDB stores the document using BSON, its binary document representation.

The MongoDB shell also accepts JavaScript-style object syntax:

```
db.students.insertOne({
  rollno: 101,
  name: "Rahul",
  course: "BCA"
})
```

Important: The command above uses JavaScript object syntax in `mongosh`. It is not strict JSON because the field names are not enclosed in double quotes. Both styles represent the same kind of document data.

What Is BSON?

BSON stands for Binary JSON.

BSON is a binary-encoded serialization format used by MongoDB to represent documents.

Although BSON is based on a JSON-like document model, it supports additional data types and stores type information.

Arjunsinh Vaghela

CORE SERVER TOOLS IN MONGODB

MongoDB provides several tools for running the database server, interacting with it, importing data, and managing backups.

5.1 `mongod`

Definition: `mongod` is the primary MongoDB database server process.

It manages database operations and handles requests from clients.

5.2 `mongosh`

Definition: `mongosh` is the MongoDB Shell, an interactive JavaScript-based environment used to connect to MongoDB and execute database commands.

MONGODB THROUGH THE JAVASCRIPT SHELL

What Is `mongosh`?

`mongosh` is an interactive shell that accepts JavaScript-based expressions and MongoDB commands.

Students can use it to perform CRUD operations and execute queries.

CRUD stands for:

- **C – Create:** Insert data.
- **R – Read:** Retrieve data.
- **U – Update:** Modify data.
- **D – Delete:** Remove data.

Starting the Shell

Open Windows CMD and execute:

```
mongosh
```

If the shell is installed and MongoDB is running at the default local address, it normally connects to the local server.

If the server is unavailable, the connection may fail. The server service must be started or the shell must be pointed to a running MongoDB instance.

Arjunsinh Vaghela